

Let's Evaluate

Project Proposal · Option B: Bring Your Own Use Case

1. Business Problem

Problem Statement

Technical hiring is broken at scale. Talent Acquisition (TA) teams at mid-to-large organisations juggle four to six disconnected tools simultaneously — an ATS, a scheduling app, a resume parser, a shared spreadsheet for interview notes, and email — just to move a single candidate from application to offer. This context-switching kills productivity and introduces errors: candidates are screened inconsistently, interviewers walk into sessions unprepared, and hiring managers have no real-time visibility into pipeline health.

Who is affected?

Persona	Pain
Recruiter / TA	Manually reads every resume; inconsistent screening criteria across the team; no audit trail
Interviewer	No structured question bank; unprepared for candidate's background; submits feedback in email or forms
TA Lead / Manager	No single view of pipeline velocity; can't measure interviewer workload or AI recommendation accuracy
Candidate	Delayed responses; receives generic communication with no context

Cost of Not Solving It

- Average time-to-hire extends by 15–20 days when screening is manual and unstructured.
- Inconsistent screening creates compliance and bias risks in regulated industries.
- Recruiter burnout: 60–80 resumes per open role, read manually.
- Lost candidates: top applicants accept competing offers while still waiting for a first call.

2. Proposed Solution

Let's Evaluate is an end-to-end AI-powered technical hiring platform that consolidates the entire hiring lifecycle into a single portal — from resume upload to final decision — without requiring any external tool.

What the system does, from the user's point of view

- **Recruiter uploads a resume** (PDF or DOCX) against an open role. The system extracts structured data, matches technologies and experience to the role's requirements, and produces a recommendation: *Proceed*, *Hold*, or *Reject* — with a reason.

- **If the AI detects ambiguous information** (e.g., a technology listed once with no context), it flags it as a "clarification needed" case, auto-drafts a personalised email, and pauses the decision until the recruiter has confirmation.
- **TA Lead assigns a panel** from the available interviewer pool, taking workload into account. The system generates role-contextualised interview questions from the candidate's resume automatically.
- **Interviewer opens the interview session**, sees the candidate's extracted profile and prepared questions, and records structured feedback live during the interview.
- **After the interview**, the TA Lead reviews consolidated feedback and records the final outcome. The pipeline view (Kanban) updates in real time.
- **Managers and leads see a dashboard** with pipeline velocity, AI recommendation agreement rate, cost per screening, and interviewer utilisation — all without building a single report.

The system is designed to be **white-labelled**: each deployment is branded per organisation (logo, colours, domain restriction) so it can be licensed as a SaaS product to other companies.

3. Must Have / Good to Have Breakdown

Category 1 — Requirement Definition (*this document*)

Must Have	Good to Have
This proposal document covering all 5 sections	Wireframe sketches or user-story map attached as appendix

Category 2 — Business Functionality

Must Have	Good to Have
End-to-end candidate lifecycle: upload → screen → assign → interview → decision	SLA / ETA tracking per stage with alerts
Role-based access control (admin, TA, TA lead, interviewer, manager, HR)	Calendar invite generation (ICS)
Candidate Kanban pipeline with status transitions	Bulk SMS / email outreach to candidates
Interview assignment with workload visibility	Multi-tenant self-service onboarding
Structured feedback collection per interview	ATS webhook integration (push outcomes to Greenhouse / Lever)

Category 3 — AI / Logic Layer

Must Have	Good to Have
Two-phase deterministic resume analysis (GPT-4o-mini extraction + GPT-4o scoring)	Fine-tuned model on organisation's historical screening decisions
Tech-match scoring with alias resolution (React ↔ ReactJS) and experience date calculation	Cross-role candidate routing suggestions ("this candidate fits Role B better")
SHA-256 resume deduplication (cached results, zero re-analysis cost for duplicates)	Automated bias-detection pass on AI reasoning output

Must Have	Good to Have
AI-generated, role-contextualised interview questions from candidate resume	Real-time AI coaching hints for interviewers during the session
Decision tree: $\geq 80\%$ match $\rightarrow$ Proceed; $\geq 60\%$ $\rightarrow$ Hold; $< 60\%$ $\rightarrow$ Reject	Confidence interval on AI recommendations
Token usage + cost telemetry per analysis	Prompt A/B testing framework
Feedback loop table tracking AI vs. recruiter vs. final outcome	Drift detection: alert when AI agreement rate drops below threshold

Category 4 — Data

Must Have	Good to Have
PostgreSQL schema: candidates, openings, roles, projects, assignments, feedback, screening results, audit log	Time-series analytics table for pipeline velocity reporting
Drizzle ORM with versioned SQL migrations (21 migrations at submission)	Data warehouse export (Parquet / CSV) for BI tools
Synthetic candidate dataset (100+ resumes) generated for demo and testing	Integration with LinkedIn Talent Insights for real market benchmarks
SHA-256 resume hash table for deduplication	GDPR-compliant data-retention policy and PII masking
Screening feedback table with AI recommendation, recruiter decision, final outcome	Anonymised, exportable dataset for model retraining

Category 5 — API

Must Have	Good to Have
RESTful API routes for all entities (candidates, openings, screenings, assignments, feedback)	OpenAPI / Swagger spec auto-generated from route handlers
/api/ai/stats — 30-day token usage, cost, cache hit rate, recommendation agreement %	Webhooks: push events (screening completed, interview assigned) to external systems
Server-side RBAC enforced on every API route	Rate limiting and API key authentication for external consumers
Zod validation on all request bodies	GraphQL layer for flexible querying by partner integrations
Public screening token API (candidate self-service endpoint)	SDK client library for white-label integrators

Category 6 — UI

Must Have	Good to Have
Next.js 16 App Router with React 19 and Tailwind CSS 4	Dark mode
Candidate Kanban pipeline (drag-and-drop via @dnd-kit)	Mobile-responsive layout

Must Have	Good to Have
Interview session view: resume summary, question bank, live notes	Rich text editor for interview feedback
AI screening wizard with step-by-step result display	Animated pipeline flow diagram (already using @xyflow/react)
Mail template builder with <code>{{placeholder}}</code> substitution	In-app notification centre (bell icon, real-time)
White-label branding (logo, org name, primary colour via env vars)	Embedded video interview (no external tool)

Category 7 — Testing

Must Have	Good to Have
Vitest unit tests covering AI scoring logic, tech matcher, resume date calculator	Mutation testing (Stryker) on business-critical logic
Playwright E2E tests: smoke, sanity, regression suites	Visual regression testing (Chromatic / Percy)
E2E coverage of full candidate lifecycle (upload → screen → assign → interview → decision)	Load testing: 50 concurrent resume analyses (k6)
CI pipeline running tests on every pull request	Contract testing for external API integrations (Pact)
Test coverage report (≥70% on AI lib)	AI output determinism tests with seeded prompts

Category 8 — Deployment

Must Have	Good to Have
Vercel deployment with environment-based config (dev / staging / prod)	Blue-green deployment with zero-downtime database migrations
Neon PostgreSQL (serverless, branching for preview environments)	Infrastructure as Code (Pulumi / Terraform)
Cloudflare R2 / AWS S3 for resume file storage	Container-based deployment option (Docker Compose) for on-premise white-label
GitHub Actions CI/CD pipeline (lint → test → build → deploy)	Automated secret rotation
Environment variable management via Vercel dashboard	Observability: OpenTelemetry traces to Datadog / Grafana

Category 9 — Demo

Must Have	Good to Have
Live walkthrough: upload resume → AI screening → assign interviewer → conduct interview → record outcome	Side-by-side comparison: manual screening time vs. AI screening time
Show AI stats dashboard (/api/ai/stats) with real cost and accuracy numbers	Demonstrate white-label branding switch live

Must Have	Good to Have
Demonstrate clarification workflow (ambiguous resume → hold → re-analyse)	Multi-role login demo (TA, interviewer, manager simultaneously)
Kanban pipeline update in real time after decision	Export job description to DOCX live

4. Data Source

Primary (Synthetic): A synthetic dataset of 120 candidate profiles will be generated programmatically using a Python script ([scripts/seed-candidates.py](#)). Each profile includes:

- Candidate name, contact info (faker-generated)
- Resume PDF with realistic employment history, technologies, and date ranges
- Role applied for (mapped to pre-seeded openings in the DB)

Generation method: [Faker.js](#) / [faker](#) (Python) for PII fields; [gpt-4o](#) with a fixed seed and deterministic temperature=0 for generating realistic but synthetic resume narrative text; [pdf-lib](#) for assembling PDFs.

Secondary (Public): Stack Overflow Developer Survey 2024 — used to calibrate the technology alias dictionary (e.g., frequency of "React" vs. "ReactJS" vs. "React.js" in real resumes) and to set realistic experience-year distributions per technology.

No real candidate PII is stored or used at any point in the demo.

5. Success Metrics

Metric	Target	How Measured
AI screening accuracy	≥ 85% agreement between AI recommendation and recruiter's final decision on the synthetic dataset	screening_feedback table: ai_recommendation vs. final_outcome
Screening time reduction	AI analysis completes in < 8 seconds per resume (vs. ~6 minutes manual read)	analysis_duration_ms column in screening_results
Resume deduplication hit rate	≥ 20% of demo submissions served from cache (demonstrating cost saving)	/api/ai/stats → cache_hit_rate
Cost per screening	< \$0.05 USD per resume analysed	Token count × model pricing, tracked in token_usage column
End-to-end cycle time	A candidate can go from upload to interview-assigned in < 5 minutes in the demo	Manual stopwatch during live demo
Test coverage	≥ 70% line coverage on src/lib/ai/	Vitest coverage report
Zero critical E2E failures	All Playwright smoke tests pass on production URL	CI pipeline green gate

Additional Considerations for Presentation

The following points strengthen the proposal under the rubric and should be included in any brief or slide deck:

Differentiation

Most ATS tools (Greenhouse, Lever, Workday) treat AI as a bolt-on feature. Let's Evaluate is **AI-first by design**: the decision logic is deterministic and auditable, not a black box, which directly addresses the bias and compliance concerns that block AI adoption in regulated industries.

Scalability Path

The white-label architecture means the same codebase can serve a 10-person startup and a 10,000-employee enterprise. The per-deployment branding, domain restriction, and role configuration are all environment-variable driven — no code changes required per customer.

Risk Mitigation

- **AI hallucination:** Mitigated by Phase 1/Phase 2 separation — GPT-4o-mini extracts facts; GPT-4o applies deterministic rules. The AI cannot invent experience years; dates are parsed deterministically from resume text.
- **Data privacy:** Synthetic data only in demo; production deployments use organisation-owned infrastructure (Neon, R2, Azure).
- **Cost overrun:** SHA-256 deduplication caches results; token telemetry surfaces cost anomalies before they compound.

Compliance Alignment

The audit log, feedback loop table, and RBAC model together satisfy the traceability requirements of EEOC (US), and provide the data lineage needed for GDPR Article 22 (automated decision-making) documentation.